ADVANCED MACHINE LEARNING FUNDAMENTALS

CASE STUDY

Predictive Maintenance for Smart Manufacturing

Using Machine Learning to predict equipment failures within 24 hours

Submitted By	Jui Tawde
Roll No.	150096725149
Cohort	Sam Altman
Subject	Advanced Machine Learning Fundamentals
Topic	Predictive Maintenance for Smart Manufacturing Using Machine Learning
Institution	ITM Skills University, Kharghar
Date of Submission	26 October 2026
Total Marks	100

TABLE OF CONTENTS

1. INTRODUCTION 3

2. PROBLEM STATEMENT 3

3. CHALLENGE 1: UNDERSTAND THE MACHINE LEARNING PROBLEM 5

4. CHALLENGE 2: DATA PREPROCESSING AND FEATURE ENGINEERING 8

5. CHALLENGE 3: HANDLE IMBALANCED DATA 11

6. CHALLENGE 4: SELECT THE BEST ALGORITHM 13

7. CHALLENGE 5: EVALUATE THE MODELS 15

8. CHALLENGE 6: OPTIMISE THE MAINTENANCE STRATEGY 20

9. CHALLENGE 7: DEPLOYMENT AND THE UNEXPECTED TWIST 24

10. PROPOSED SYSTEM ARCHITECTURE 28

11. EXPECTED BENEFITS 29

12. LIMITATIONS AND RISK MANAGEMENT 30

13. CONCLUSION 30

1. INTRODUCTION

Predictive maintenance is a machine learning application that predicts potential equipment failures before they occur. It helps manufacturing companies reduce unexpected breakdowns, minimize maintenance expenses, improve production efficiency, and increase the lifespan of industrial machines.

In this case study, an automobile manufacturing company operates 500 industrial machines across three factories. These machines include robotic arms, hydraulic presses, conveyor motors, and CNC machines.

Currently, the company follows a fixed maintenance schedule based on operating hours rather than the actual condition of each machine. This approach results in unexpected failures and unnecessary maintenance interventions.

The company wants to develop a machine learning system that predicts whether a machine is likely to fail within the next 24 hours. This will allow maintenance technicians to take preventive action before a breakdown disrupts production.

Objective

The primary objectives of this case study are:

1. To design a machine learning system that predicts machine failures within the next 24 hours.
2. To preprocess sensor data and engineer meaningful features for better predictions.
3. To handle imbalanced data and select suitable machine learning algorithms.
4. To evaluate models using appropriate performance metrics.
5. To develop a cost-effective maintenance decision policy.
6. To deploy, monitor, and maintain the predictive system in a real manufacturing environment.

2. PROBLEM STATEMENT

The manufacturing company operates 500 machines across three factories and has collected data from 10,000 machine operating cycles.

The existing fixed-schedule maintenance approach creates two major problems:

- **Unexpected machine failures:** Machines may break down before their scheduled maintenance, resulting in production downtime and financial losses.
- **Unnecessary maintenance:** Machines that are still functioning properly may be serviced too frequently, wasting technician time, spare parts, and money.

The proposed solution is to develop a supervised machine learning classification model that predicts whether a machine will experience a failure within the next 24 hours.

The model will analyze historical operating data, sensor readings, machine characteristics, and maintenance information to generate failure probabilities. These predictions will help the company prioritize maintenance activities according to risk.

2.1 Dataset Description

The dataset contains 10,000 operating cycles and the following features.

Feature	Description
machine_id	Unique identifier for each machine
machine_type	Type of industrial machine
operating_hours	Total operating hours of the machine
temperature	Operating temperature in °C
vibration	Vibration level measured by sensors
rotational_speed	Rotational speed in RPM
pressure	Operating pressure
power_consumption	Electrical power consumption
maintenance_days_ago	Number of days since the last maintenance
error_count_24h	Number of recorded errors in the last 24 hours
load_percentage	Percentage of machine capacity being used
failure_within_24h	Target variable: 1 if the machine fails within 24 hours, otherwise 0

2.2 Initial Dataset Analysis

Dataset characteristic	Value
Total operating cycles	10,000
Failure cycles	300
Non-failure cycles	9,700
Missing sensor readings	4%
Failure rate	3%
Non-failure rate	97%

The dataset has a significant class imbalance because only 3% of the operating cycles are associated with failures. It also contains missing sensor readings, potentially incorrect extreme values, incomplete maintenance records, and machines with different failure rates.

These issues must be addressed before training a reliable model.

3. CHALLENGE 1: UNDERSTAND THE MACHINE LEARNING PROBLEM

Q1. Is this a classification, regression, or clustering problem? Justify your answer.

Answer

This is a **binary classification problem** because the model must predict one of two possible outcomes:

- `1`: The machine fails within the next 24 hours.
- `0`: The machine does not fail within the next 24 hours.

The target variable, `failure_within_24h`, contains two discrete classes rather than a continuous numerical value.

Regression would be appropriate if the company wanted to predict a continuous value, such as the number of hours remaining before failure. Clustering could group machines according to similar operating characteristics without predefined failure labels.

Therefore, binary classification is the most appropriate approach for the given problem.

Q2. Is supervised learning appropriate for this task? Why?

Answer

Yes, supervised learning is appropriate because the dataset contains historical operating records and a labeled target variable, `failure_within_24h`.

The model can learn the relationship between machine operating conditions and known failure outcomes.

For example, it may learn that a combination of increasing vibration, unusually high temperature, frequent errors, and heavy operating loads is associated with a higher probability of failure.

After training on labeled examples, the model can predict the probability of failure for new operating cycles.

However, the labels must be reliable, and the historical records must contain enough examples of actual failures for the model to learn useful patterns.

Q3. What are the input features and the target variable?

Answer

The input features are the information supplied to the model to make predictions.

The available input features are:

1. `machine_id`
2. `machine_type`
3. `operating_hours`
4. `temperature`
5. `vibration`

6. `rotational_speed`
7. `pressure`
8. `power_consumption`
9. `maintenance_days_ago`
10. `error_count_24h`
11. `load_percentage`

The target variable is `failure_within_24h`.

It indicates whether the machine experiences a failure within the following 24 hours.

Although `machine_id` is present in the dataset, it should not automatically be used as a predictive feature. Its suitability must be evaluated because a unique identifier may encourage the model to memorize machine-specific patterns rather than learn generalizable failure behavior.

The target must also be constructed using only future failure outcomes relative to the prediction timestamp, while input features must contain only information available at that timestamp.

Q4. Why might traditional scheduled maintenance be less efficient than predictive maintenance?

Answer

Traditional scheduled maintenance is based on fixed operating hours or predetermined time intervals. It does not necessarily consider the actual condition of a machine.

This creates two problems. A machine may fail before its scheduled maintenance, causing unexpected downtime. Alternatively, a machine that is functioning correctly may be serviced unnecessarily.

Predictive maintenance uses sensor readings, operating conditions, error history, and maintenance information to estimate failure risk.

Its major advantages include:

- Reduced unexpected breakdowns.
- Lower unnecessary maintenance expenses.
- Better utilization of technicians and spare parts.
- Improved production continuity.
- More effective maintenance scheduling.
- Better visibility into machine health.

Therefore, predictive maintenance can make maintenance activities more targeted and cost-effective, provided the model is sufficiently reliable and its predictions are translated into appropriate operational actions.

Q5. Could unsupervised learning help identify abnormal machine behaviour even when no failure labels are available?

Answer

Yes. Unsupervised learning can identify unusual machine behavior when labeled failure examples are unavailable.

Algorithms such as Isolation Forest, clustering-based anomaly detection, and autoencoders can learn patterns in normal operating data and identify observations that differ significantly from those patterns.

For example, a machine may show an unusual combination of vibration, temperature, and power consumption even when no confirmed failure label exists.

The system can flag such observations for inspection.

However, an anomaly does not necessarily indicate an upcoming failure. Some unusual readings may result from legitimate changes in production load or operating conditions.

Therefore, unsupervised anomaly detection should complement supervised failure prediction rather than replace it entirely.

Q6. What assumptions are required for the model to predict failures 24 hours in advance?

Answer

The model depends on several assumptions:

1. **Reliable historical labels:** Previous records must correctly indicate whether a failure occurred within 24 hours after the prediction time.
2. **Representative data:** Training data should reflect the operating conditions, machine types, and failure patterns expected during deployment.
3. **Available input features:** Sensor readings and other features must be available before the prediction is made.
4. **Consistent measurements:** Sensors should be calibrated, and measurement units should remain consistent.
5. **Meaningful relationships:** Machine operating conditions must contain patterns associated with future failures.
6. **Limited distribution changes:** Operating conditions should not change so substantially that the learned relationships become unreliable without adaptation.
7. **Correct prediction timing:** The system must generate predictions early enough for technicians to intervene before failure.
8. **Actionable maintenance:** Suitable technicians, spare parts, and maintenance procedures must be available to respond to high-risk predictions.

The model predicts risk rather than guaranteeing that a machine will or will not fail. Its performance must therefore be evaluated continuously.

4. CHALLENGE 2: DATA PREPROCESSING AND FEATURE ENGINEERING

Q1. How would you determine whether extreme sensor values are genuine anomalies or sensor errors?

Answer

I would investigate extreme sensor readings before removing or modifying them.

First, I would compare the readings with the physically possible operating ranges of the relevant machine type. For example, a temperature of -25°C may be suspicious for an operating industrial machine, but its validity depends on the machine, sensor, and operating environment.

Second, I would inspect the sensor's historical readings and compare the observation with nearby measurements. A sudden isolated spike in vibration followed immediately by normal values may indicate a sensor malfunction.

Third, I would compare the observation with other available information, such as rotational speed, load percentage, power consumption, maintenance logs, and error counts.

Fourth, I would consult sensor specifications and maintenance technicians to determine whether the reading is physically plausible.

Based on this investigation:

- Confirmed sensor errors should be corrected when a reliable correction is available or marked as missing.
- Genuine extreme operating conditions should be retained because they may indicate an impending failure.
- Uncertain observations should be flagged for further investigation rather than automatically deleted.

This approach prevents the loss of important failure-related information.

Q2. Would you remove rows with missing values, replace them with mean or median values, or use another approach?

Answer

I would use a feature-specific missing-value strategy instead of removing every row containing missing data.

The dataset has 4% missing sensor readings. Removing all affected records could reduce the amount of training data and potentially eliminate important failure examples.

For numerical features such as temperature, vibration, pressure, and power consumption, I would generally use median imputation when appropriate. The median is less affected by extreme values than the mean.

For categorical features such as `machine_type`, I would use the most frequent category or an explicit unknown category when appropriate.

For missing maintenance records, I would not automatically assume that the machine has never been maintained. I would introduce a missing-history indicator and use a suitable imputation strategy based on the available records.

I would also consider time-based interpolation for sensor readings when the time series supports it and interpolation does not use future information that would be unavailable at prediction time.

All imputation values and preprocessing parameters must be learned from the training data only. The same fitted preprocessing steps must then be applied to validation and test data.

Q3. Should `machine_id` be used as a model feature? Explain the risks.

Answer

I would not directly use `machine_id` as an ordinary numerical feature because the identifier does not represent a meaningful physical measurement.

For example, machine 101 is not necessarily more failure-prone than machine 100 simply because its identifier is larger.

Using machine IDs as numerical inputs may create misleading relationships. One-hot encoding a large number of unique identifiers can also increase dimensionality and encourage memorization.

However, machine identity may still be useful for grouping records, tracking operating histories, calculating machine-specific failure rates, and designing leakage-resistant data splits.

If machine-specific historical information is used as a predictive feature, it must be calculated only from information available before the prediction timestamp.

Machine type and relevant historical behavior are generally more useful than the raw identifier itself.

Q4. How would you encode `machine_type`?

Answer

I would use one-hot encoding for `machine_type` as a suitable initial approach.

For example, suppose the dataset contains four machine types: robotic arm, hydraulic press, conveyor motor, and CNC machine.

One-hot encoding would create separate binary columns:

Machine type	Robotic arm	Hydraulic press	Conveyor motor	CNC machine
Robotic arm	1	0	0	0
Hydraulic press	0	1	0	0
Conveyor motor	0	0	1	0
CNC machine	0	0	0	1

This approach does not introduce an artificial ranking between machine categories.

For algorithms that support categorical features directly, native categorical handling may also be considered.

If a new machine type appears after deployment, the system should handle the unknown category safely rather than fail during prediction.

Q5. Propose at least three new features that could improve predictions.

Answer

I would create the following additional features:

1. Vibration change over the past hour

This feature measures the change in vibration over the previous hour.

> $\text{Vibration Change} = V_{\text{current}} - V_{\text{one hour ago}}$

A rapid increase may indicate bearing wear, imbalance, or another developing mechanical problem.

2. Temperature change over the past hour

This feature measures the difference between the current temperature and the temperature recorded one hour earlier.

> $\text{Temperature Change} = T_{\text{current}} - T_{\text{one hour ago}}$

A sudden temperature increase may indicate overheating or excessive friction.

3. Rolling error count

This feature calculates the total number of recorded errors over a recent time window, such as the previous six hours.

Repeated errors may indicate an increasingly unstable machine condition.

4. Time since the previous failure

This feature measures the time elapsed since the machine's most recent confirmed failure. It may help identify recurring problems, provided the historical information is available at prediction time.

5. Rolling average of vibration

This feature calculates the average vibration over a recent period, such as one hour. It helps distinguish a persistent increase in vibration from a single noisy measurement.

These features can capture trends and changes that a single sensor reading may not reveal.

All time-based features must be constructed chronologically and without using future observations.

Q6. Explain why randomly splitting individual rows might create data leakage when records from the same machine appear in both training and testing datasets.

Answer

Randomly splitting individual rows may place records from the same machine in both the training and test datasets.

Records from a single machine can have similar operating patterns, sensor characteristics, maintenance histories, and failure behavior. Consequently, the model may appear to perform well on test records because it has already learned patterns specific to that machine.

This can produce an overly optimistic estimate of real-world performance, particularly when the model must predict failures for previously unseen machines.

To reduce this risk, I would use a group-based split in which records from selected machines are reserved for testing when evaluating generalization to unseen machines.

If the intended deployment primarily predicts future failures for machines already being monitored, I would also use a chronological split, training on earlier periods and testing on later periods.

The choice depends on the intended deployment scenario. In either case, preprocessing and feature engineering must respect the split boundaries.

5. CHALLENGE 3: HANDLE IMBALANCED DATA

Q1. Why is this model unacceptable despite its high accuracy?

Answer

The model is unacceptable because it predicts that every machine will operate normally and detects zero failures.

The dataset contains 3% failure cycles and 97% non-failure cycles. A model that always predicts the majority class can therefore achieve 97% accuracy without identifying a single actual failure.

Its recall for the failure class is 0%, meaning it fails to detect all actual failures.

In manufacturing, missing an impending failure may cause production downtime, equipment damage, and substantial financial losses.

Therefore, accuracy alone is not a sufficient performance measure for this problem. The model must be evaluated based on its ability to detect failures and the operational cost of its predictions.

Q2. Would you use class weights, oversampling, undersampling, or SMOTE?

Answer

I would begin by evaluating class-weighted learning and then compare it with carefully applied oversampling and SMOTE.

Class weighting: I would assign a higher penalty to incorrectly classifying failure cases. This encourages the model to pay more attention to the minority class without creating synthetic observations.

Oversampling: I would increase the representation of failure examples in the training data. Simple random oversampling can help the model learn the minority class but may increase overfitting because existing failure examples are repeated.

Undersampling: I would reduce the number of non-failure examples in the training data. This can reduce training time and improve the relative representation of failures, but it may discard useful information about normal machine behavior.

SMOTE: I would consider generating synthetic failure examples by interpolating between minority-class observations. This can help when failure examples are scarce, but the synthetic observations may not represent physically realistic machine conditions.

My preferred initial approach would be class weighting because it preserves the original training observations. I would then compare it with oversampling and SMOTE using validation data and the same evaluation metrics.

Any resampling must be performed only within the training data, not before splitting the dataset.

Q3. What are the disadvantages of each approach?

Answer

Technique	Disadvantages
Class weighting	May increase false positives or reduce probability calibration; suitable weights require validation.
Oversampling	Can cause overfitting because duplicated examples may be memorized.
Undersampling	Removes normal operating examples and may discard useful information.
SMOTE	May generate unrealistic synthetic examples, especially when failure patterns involve complex time-dependent sensor relationships.

No technique is guaranteed to improve every model. The best approach should be selected using validation results and manufacturing costs.

Q4. Should the test dataset be artificially balanced?

Answer

No. The final test dataset should generally preserve the natural class distribution expected in real manufacturing operations.

If the original operating environment has a failure rate of approximately 3%, the test set should represent that prevalence as closely as practical.

Artificially balancing the test dataset can distort precision, false-positive burden, and the estimated number of maintenance interventions required in production.

A separate balanced diagnostic dataset may be useful for certain controlled comparisons, but it should not replace evaluation on a representative test set.

The test set must remain untouched during model selection and hyperparameter tuning.

Q5. Which evaluation metrics would you use instead of relying on accuracy alone?

Answer

I would use the following metrics:

1. **Precision:** Measures the proportion of predicted failures that are actual failures. It helps assess how many maintenance alerts are justified.
2. **Recall:** Measures the proportion of actual failures detected by the model. High recall is important because missed failures may cause expensive downtime.

3. **F1-score:** Combines precision and recall into a single metric using their harmonic mean.
4. **False positive rate:** Measures the proportion of actual non-failure cases incorrectly flagged as failures.
5. **Precision-recall AUC:** Evaluates the trade-off between precision and recall across different decision thresholds and is particularly useful for imbalanced datasets.
6. **ROC-AUC:** Measures the model's ability to rank failure cases above non-failure cases across thresholds.
7. **Expected operational cost:** Estimates the financial impact of missed failures, maintenance interventions, and unnecessary alerts.

I would prioritize recall and precision-recall performance while also considering false-positive burden and total expected cost. The final choice would depend on the consequences of missed failures and unnecessary maintenance.

6. CHALLENGE 4: SELECT THE BEST ALGORITHM

Q1. Select one baseline algorithm and one advanced candidate.

Answer

I would select **Logistic Regression as the baseline algorithm** and **Gradient Boosting as the advanced candidate**.

Logistic Regression provides a simple, interpretable baseline, while Gradient Boosting can learn more complex nonlinear relationships among sensor readings and failure outcomes.

Q2. Explain why your selected algorithms are suitable.

Answer

A. Baseline: Logistic Regression

Logistic Regression estimates the probability that a machine will fail within the next 24 hours.

Its advantages include:

- Simple implementation and relatively fast training.
- Probability estimates that are useful for risk-based decisions.
- Interpretability through feature coefficients.
- Support for class weighting.
- A useful baseline for comparing more complex models.

However, Logistic Regression may struggle with complex nonlinear relationships unless appropriate feature transformations and interaction features are introduced.

B. Advanced Candidate: Gradient Boosting

Gradient Boosting builds an ensemble of decision trees sequentially, with each new tree attempting to improve the errors made by the existing ensemble.

Its advantages include:

- Ability to model nonlinear relationships.

- Ability to capture interactions between operating conditions.
- Strong predictive performance on many structured datasets.
- Ability to incorporate informative engineered features.
- Support for feature-importance analysis in suitable implementations.

For example, the relationship between vibration and failure risk may depend on temperature, machine type, and load percentage. Gradient Boosting can learn such interactions without requiring every relationship to be manually specified.

However, it requires careful hyperparameter tuning, validation, and monitoring for overfitting.

A Random Forest would also be a strong candidate for comparison, but Gradient Boosting is my selected advanced model for this case study.

Q3. Identify which algorithms may benefit from feature scaling.

Answer

Logistic Regression generally benefits from feature scaling, especially when its numerical inputs have very different units and magnitudes.

For example, operating hours may be measured in thousands, whereas temperature may be measured in tens. Standardization can improve optimization and make regularization behave more consistently.

Feature scaling is also important for algorithms that rely on distances or gradient-based optimization, such as k-nearest neighbors, support vector machines, and neural networks.

Decision Trees, Random Forests, and most Gradient Boosting implementations based on decision trees generally do not require feature scaling because their splits depend on feature ordering and thresholds rather than feature magnitudes.

Categorical encoding and missing-value handling may still be required for tree-based algorithms.

Q4. Explain how you would detect overfitting.

Answer

Overfitting occurs when a model learns the training data too closely, including noise and accidental patterns, and consequently performs poorly on unseen data.

I would detect it by comparing performance on the training set and validation set.

For example, if a model achieves 99% recall on training data but only 55% recall on validation data, the large performance gap may indicate overfitting.

I would also examine:

- Training and validation loss.
- Precision, recall, F1-score, and precision-recall AUC on separate datasets.
- Performance across different machine groups and time periods.
- Sensitivity to changes in hyperparameters.
- Tree depth, number of leaves, and model complexity.
- Whether the model relies excessively on machine-specific or leakage-prone features.

To reduce overfitting, I would use regularization, limit model complexity, tune hyperparameters using validation data, and collect more representative failure examples where possible.

Q5. Describe how you would tune model hyperparameters without contaminating the test set.

Answer

I would divide the available data into training, validation, and final test sets.

The training set would be used to fit models. The validation set or cross-validation folds would be used to select hyperparameters and compare model configurations. The test set would be reserved for the final evaluation.

For Logistic Regression, I would tune parameters such as regularization strength and class weights.

For Gradient Boosting, I would tune:

- Number of boosting iterations or trees.
- Learning rate.
- Maximum tree depth or number of leaves.
- Minimum samples required for a split or leaf.
- Row or feature subsampling, where supported.
- Regularization parameters, where supported.

I would use group-aware or time-aware cross-validation according to the deployment objective.

All preprocessing, imputation, feature selection, and resampling steps would be fitted separately within each training fold.

After selecting the final configuration, I would evaluate it once on the untouched test set and report the resulting performance metrics.

7. CHALLENGE 5: EVALUATE THE MODELS

The case study provides the following results for two models evaluated on 2,000 operating cycles. There are 60 failure cycles and 1,940 non-failure cycles.

Q1. Calculate accuracy, precision, recall, F1-score, and false positive rate for both models.

Answer

Formulas

1. Accuracy

Accuracy measures the proportion of all predictions that are correct.

> Accuracy = (TP + TN) / (TP + TN + FP + FN)

2. Precision

Precision measures the proportion of predicted failures that are actual failures.

> Precision = TP / (TP + FP)

3. Recall

Recall measures the proportion of actual failures correctly detected.

> Recall = TP / (TP + FN)

4. F1-score

F1-score combines precision and recall.

> F1-score = 2 × Precision × Recall / (Precision + Recall)

5. False Positive Rate

False positive rate measures the proportion of actual non-failure cases incorrectly classified as failures.

> False positive rate = FP / (FP + TN)

Model A Calculations

Given:

- True Positives (TP) = 45
- False Negatives (FN) = 15
- False Positives (FP) = 100
- True Negatives (TN) = 1,840

1. Accuracy

> Accuracy = (45 + 1,840) / 2,000 = 1,885 / 2,000 = **94.25%**

2. Precision

> Precision = 45 / (45 + 100) = 45 / 145 = **31.03%**

3. Recall

> Recall = 45 / (45 + 15) = 45 / 60 = **75.00%**

4. F1-score

> F1-score = 2 × 0.3103 × 0.75 / (0.3103 + 0.75) ≈ 0.4389 = **43.89%**

5. False Positive Rate

> False positive rate = 100 / (100 + 1,840) = 100 / 1,940 ≈ **5.15%**

Model B Calculations

Given:

- True Positives (TP) = 54
- False Negatives (FN) = 6
- False Positives (FP) = 250
- True Negatives (TN) = 1,690

1. Accuracy

> Accuracy = (54 + 1,690) / 2,000 = 1,744 / 2,000 = **87.20%**

2. Precision

> Precision = 54 / (54 + 250) = 54 / 304 ≈ **17.76%**

3. Recall

> Recall = 54 / (54 + 6) = 54 / 60 = **90.00%**

4. F1-score

> F1-score = 2 × 0.1776 × 0.90 / (0.1776 + 0.90) ≈ 0.2965 = **29.65%**

5. False Positive Rate

> False positive rate = 250 / (250 + 1,690) = 250 / 1,940 ≈ **12.89%**

Comparison of Results

Evaluation metric	Model A	Model B
True positives	45	54
False negatives	15	6
False positives	100	250
True negatives	1,840	1,690
Accuracy	94.25%	87.20%
Precision	31.03%	17.76%
Recall	75.00%	90.00%
F1-score	43.89%	29.65%

Evaluation metric	Model A	Model B
False positive rate	5.15%	12.89%

Note: Metrics are calculated from the confusion-matrix counts supplied in the case study. Small differences may arise from rounding.

Q2. Which model detects more actual failures?

Answer

Model B detects more actual failures.

Model A correctly detects 45 of the 60 failure cycles, whereas Model B detects 54.

Therefore, Model B detects nine additional failures and reduces missed failures from 15 to 6.

Its recall is 90%, compared with 75% for Model A.

This makes Model B more effective at identifying machines that are actually at risk of failure.

Q3. Which model generates more unnecessary maintenance alerts?

Answer

Model B generates more unnecessary maintenance alerts.

Model A produces 100 false positives, whereas Model B produces 250 false positives.

Therefore, Model B generates 150 additional false-positive alerts.

This can increase maintenance expenses, technician workload, spare-parts consumption, and unnecessary interruptions to production.

Q4. Is the model with higher recall necessarily the better choice?

Answer

No. Higher recall alone does not guarantee that a model is better.

Model B has higher recall and misses fewer actual failures, but it also generates substantially more false positives and has lower precision.

Model A has higher precision and accuracy, meaning its alerts are more reliable overall, but it misses more actual failures.

The better model depends on the relative cost of missed failures, unnecessary maintenance, production downtime, and available maintenance resources.

Therefore, the models should be compared using operational costs in addition to predictive performance metrics.

Q5. How would you quantify the financial impact of false negatives versus false positives?

Answer

I would assign an estimated financial cost to each type of error.

According to the case study:

- Unexpected machine failure costs ₹5,00,000.
- Preventive maintenance costs ₹20,000.
- Unnecessary maintenance costs ₹20,000.

A false negative represents a failure that the model does not detect in advance. A false positive represents an unnecessary maintenance intervention.

Using the stated failure and unnecessary-maintenance costs, the direct error costs can be estimated as follows.

Model A:

> False-negative cost = 15 × ₹5,00,000 = ₹75,00,000
> False-positive cost = 100 × ₹20,000 = ₹20,00,000
> Total estimated error cost = **₹95,00,000**

Model B:

> False-negative cost = 6 × ₹5,00,000 = ₹30,00,000
> False-positive cost = 250 × ₹20,000 = ₹50,00,000
> Total estimated error cost = **₹80,00,000**

Cost component	Model A	Model B
False-negative cost	₹75,00,000	₹30,00,000
False-positive cost	₹20,00,000	₹50,00,000
Total estimated error cost	₹95,00,000	₹80,00,000

Under this simplified calculation, Model B has an estimated error cost ₹15,00,000 lower than Model A.

This calculation assumes that each false negative corresponds to one failure with the stated cost and each false positive corresponds to one unnecessary intervention. It does not include additional operating costs, intervention costs for correctly detected failures, or other financial consequences.

A more complete cost analysis should include the cost of every maintenance intervention and the expected failures avoided through timely maintenance.

Q6. Which model would you recommend if every unexpected machine failure caused four hours of production downtime?

Answer

I would initially recommend **Model B if avoiding unexpected failures is the primary objective**, subject to validation of its operational costs and maintenance capacity.

Model B detects nine additional failures compared with Model A. If each missed failure causes four hours of downtime, this represents a potential reduction of:

> 9 × 4 hours = **36 hours** of production downtime across the evaluated cases

Model B also reduces the number of missed failures from 15 to 6. However, it produces 150 additional false-positive alerts.

The company must determine whether the value of avoiding additional downtime exceeds the cost of the extra maintenance interventions and associated production disruption.

Using the simplified cost calculation above, Model B has an estimated net advantage of ₹15,00,000 before accounting for additional intervention costs for true-positive cases and other operational effects.

Therefore, Model B is the stronger initial choice when failure prevention is prioritized, but I would validate the decision using real downtime costs, maintenance capacity, and an appropriate operating threshold before deploying it.

8. CHALLENGE 6: OPTIMISE THE MAINTENANCE STRATEGY

The case study provides the following illustrative costs:

Event	Estimated cost
Unexpected machine failure	₹5,00,000
Preventive maintenance intervention	₹20,000
Unnecessary maintenance intervention	₹20,000

Q1. Should a machine be serviced whenever the model predicts a failure probability above 50%?

Answer

No. A fixed threshold of 50% should not automatically be used for every machine.

The appropriate threshold depends on the cost of a missed failure, the cost of maintenance, the reliability of the model's probabilities, and the company's operational capacity.

If a failure costs ₹5,00,000 and preventive maintenance costs ₹20,000, even a relatively low probability of failure may justify intervention.

Assuming the model's probability estimate is calibrated and maintenance completely prevents the relevant failure, the simplified expected cost of not intervening is:

> Expected failure cost = Probability of failure × ₹5,00,000

The intervention is economically preferable when:

> Probability of failure × ₹5,00,000 > ₹20,000

Therefore:

> Probability of failure > ₹20,000 / ₹5,00,000
> Break-even probability = **4%**

This is a simplified theoretical threshold, not an automatic production setting. The actual decision must account for intervention effectiveness, probability calibration, downtime consequences, spare-parts availability, and other operating costs.

Q2. What happens if you lower the decision threshold to 20%?

Answer

Lowering the decision threshold to 20% means that more machines will be flagged for maintenance or further investigation.

The likely effects are:

- Higher recall because more potential failures are detected.
- Fewer false negatives.
- More false positives.
- Increased maintenance workload.
- Higher expenditure on inspections and interventions.
- Potentially reduced unexpected downtime.

A 20% threshold may be useful when missed failures are especially costly. However, it may also result in unnecessary maintenance if the model's predictions are poorly calibrated or many flagged machines would have continued operating normally.

Q3. What happens if you increase the threshold to 80%?

Answer

Increasing the threshold to 80% means that maintenance is triggered only when the predicted probability of failure is very high.

The likely effects are:

- Fewer maintenance alerts.
- Fewer false positives.
- Lower unnecessary-maintenance expenditure.
- Lower maintenance-team workload.
- Potentially higher precision.
- More false negatives and lower recall.

This approach may reduce unnecessary interventions, but it also increases the risk of missing failures that the model assigns a probability between 20% and 80%.

Consequently, an 80% threshold may be unsuitable when unexpected failures cause substantial financial losses.

Q4. How would you select a threshold based on the costs of missed failures and unnecessary maintenance?

Answer

I would evaluate several candidate thresholds using validation data and calculate the expected financial cost associated with each threshold.

For each candidate threshold, I would estimate:

1. The number of true positives.
2. The number of false negatives.
3. The number of false positives.
4. The number of true negatives.
5. The expected cost of unexpected failures.
6. The cost of maintenance interventions.
7. The resulting total operational cost.

I would select a threshold that minimizes expected cost while meeting practical constraints such as maintenance-team capacity and spare-parts availability.

Under the simplified assumptions that a failure costs ₹5,00,000, preventive maintenance costs ₹20,000, and maintenance completely prevents the relevant failure, the break-even probability is 4%.

However, this threshold should not be applied blindly. If preventive maintenance only partially reduces failure risk, or if a machine requires different maintenance procedures, the optimal threshold will change.

I would also verify probability calibration because a predicted probability of 20% should correspond approximately to a 20% failure frequency among comparable predictions.

Q5. Why should the company consider maintenance-team capacity and spare-parts availability?

Answer

A predictive model may identify many machines as high risk at the same time, especially during high-production periods.

If the maintenance team cannot service all flagged machines promptly, the company may experience delays even though the model identifies potential problems correctly.

Similarly, the required spare parts may not be available when an intervention is recommended.

Ignoring these constraints can lead to excessive alerts, poor prioritization, longer waiting times, and unnecessary interruptions to production.

Therefore, the maintenance strategy should consider:

- The number of available technicians.
- The urgency and severity of each predicted failure.
- Spare-parts inventory.
- Maintenance duration.
- Production schedules.

- Machine criticality.
- The expected cost of delaying intervention.

The system should prioritize high-risk and high-impact machines instead of treating every alert equally.

Q6. Develop a decision policy with at least three levels: normal operation, increased monitoring, and urgent maintenance.

Answer

I would implement the following risk-based decision policy.

Risk level	Illustrative probability	Recommended action
Normal operation	Below 20%	Continue normal operations and routine monitoring.
Increased monitoring	20% to below 80%	Increase sensor monitoring frequency, review recent trends, inspect error logs, and schedule a condition-based inspection.
Urgent maintenance	80% or above	Prioritize technician assessment, verify the warning, prepare necessary spare parts, and arrange urgent maintenance when justified.

These thresholds are illustrative starting points rather than proven optimal values. The final thresholds should be calibrated using validation results and expected operational costs. In particular, the simplified 4% economic break-even calculation indicates that lower-risk cases may also justify intervention under the stated assumptions.

The company should therefore distinguish between automatic maintenance, increased monitoring, and urgent human review rather than using a single probability threshold for every action.

Additional rules should include:

- A critical machine fault or independently verified unsafe condition may override the model's probability-based category.
- Repeated high-risk predictions should increase priority.
- Technicians should verify high-impact alerts before performing costly interventions when operationally safe.
- The system should account for maintenance capacity and spare-parts availability.
- Every intervention and subsequent machine outcome should be recorded to improve future evaluation.

This policy balances failure prevention, unnecessary maintenance, and operational feasibility.

9. CHALLENGE 7: DEPLOYMENT AND THE UNEXPECTED TWIST

Six months after deployment, the number of unexpected failures increases even though the model software has not changed.

The investigation reveals that new machine models have been introduced, production loads have increased, sensor calibration has changed, new component defects have appeared, and false alarms have increased during high-production periods.

Q1. Explain the difference between data drift and concept drift.

Answer

Data drift occurs when the distribution of the model's input features changes over time.

For example, the factory may begin operating machines at higher loads, introducing a different distribution of `load_percentage`, temperature, and power consumption.

Concept drift occurs when the relationship between the input features and the target variable changes.

For example, a new component defect may cause failures under operating conditions that previously appeared normal. The relationship between sensor readings and the probability of failure has therefore changed.

Data drift concerns changes in the input data distribution, whereas concept drift concerns changes in the relationship between inputs and outcomes.

Both can reduce model performance after deployment.

Q2. Which observations could cause each type of drift?

Answer

Observation	Potential type of drift	Explanation
New machine models introduced	Data drift; potentially concept drift	Their sensor characteristics and operating patterns may differ from the training machines, and their failure relationships may also differ.
Higher production loads	Data drift; potentially concept drift	Load distributions change, and failure risk may respond differently under sustained heavy loads.
Sensor calibration changes	Data drift	Measured temperature, vibration, or other values may shift even when the physical condition has not changed.
New component defects	Concept drift; potentially data drift	Previously unseen failure mechanisms may alter the relationship between the observed features and failures.

Observation	Potential type of drift	Explanation
More false alarms during high-production periods	Possible drift or threshold mismatch	The input distribution, probability calibration, or relationship between operating conditions and failures may have changed.

These classifications are not mutually exclusive. The same event can produce both data drift and concept drift.

For example, new machine models may introduce different sensor distributions and also exhibit different failure behavior.

Q3. How would you determine whether the model is deteriorating?

Answer

I would establish a continuous monitoring system that tracks both input data and predictive performance.

First, I would monitor the distributions of important features such as temperature, vibration, rotational speed, pressure, power consumption, and load percentage.

Second, I would track data-quality indicators, including missing-value rates, sensor failures, impossible readings, and calibration changes.

Third, I would compare model predictions with confirmed machine outcomes once sufficient time has passed to establish whether a failure occurred within the 24-hour prediction window.

The key performance indicators would include:

- Recall and false-negative rate.
- Precision and false-positive rate.
- F1-score and precision-recall AUC.
- Probability calibration.
- Number of maintenance alerts.
- Confirmed failures after predictions.
- Unexpected breakdowns and associated downtime.
- Maintenance cost per machine.
- Performance by machine type and factory.

I would compare these measurements with the model's original baseline and predefined operational limits.

If performance deteriorates significantly, I would investigate whether the cause is data drift, concept drift, sensor problems, incorrect labels, or an inappropriate decision threshold.

Q4. Would you immediately retrain the model using the newest data?

Answer

No. I would not immediately retrain the model using all the newest data without investigation.

Recent data may contain sensor errors, incomplete failure labels, incorrect timestamps, or observations from machine types that the current model does not support.

Blind retraining could teach the model incorrect relationships or reduce its performance on previously supported machines.

Instead, I would first:

1. Investigate the causes of the increased failures and false alarms.
2. Verify sensor calibration and data quality.
3. Confirm the accuracy of recent failure labels.
4. Determine whether the model's input distributions or failure relationships have changed.
5. Build a representative dataset containing recent operating conditions and confirmed outcomes.
6. Train candidate models and compare them with the existing model.
7. Validate the candidates before deployment.

The existing model should remain in service only if it continues to satisfy the company's safety and performance requirements. If its reliability becomes unacceptable, the company should switch to an approved fallback process while the model is investigated.

Q5. How would you design a monitoring and retraining strategy?

Answer

I would implement a continuous monitoring and controlled retraining process.

Step 1: Monitor incoming data

Track sensor readings, missing values, extreme observations, data distributions, and changes in machine types.

Step 2: Monitor prediction behavior

Track the number of alerts, predicted failure probabilities, changes in alert frequency, and the proportion of machines assigned to each risk level.

Step 3: Collect verified outcomes

Record actual machine failures, maintenance interventions, sensor replacements, and post-maintenance outcomes. Ensure that labels correspond to the correct prediction windows.

Step 4: Detect performance deterioration

Compare recall, precision, false-negative rates, calibration, and operational costs against predefined limits.

Step 5: Investigate the cause

Determine whether deterioration results from data drift, concept drift, sensor calibration, new machine types, or changes in production operations.

Step 6: Retrain candidate models

Use validated recent data together with appropriate historical data. Preserve representative examples of different machine types and operating conditions.

Step 7: Validate before deployment

Evaluate the candidate model on time-separated and machine-appropriate holdout data. Compare it with the existing model and confirm that it meets performance and operational-cost requirements.

Step 8: Deploy gradually

Use shadow deployment or a controlled pilot to compare predictions before replacing the production model.

Step 9: Maintain rollback capability

Keep the previous approved model and configuration available so that the company can revert if the new model performs poorly.

Step 10: Repeat the process

Continue monitoring and periodically reassess the model as machine conditions, production requirements, and failure mechanisms evolve.

Retraining should be triggered by evidence of deterioration or meaningful changes in operating conditions, not simply by the passage of time.

Q6. How would you validate the model on new machine types?

Answer

I would validate the model specifically on the new machine types before allowing its predictions to control maintenance decisions.

First, I would collect representative operating data for the new machines, including sensor readings, operating loads, maintenance history, and confirmed failure outcomes.

Second, I would verify that the features use compatible units, definitions, and measurement ranges. New machines may require additional features or different operating limits.

Third, I would evaluate performance separately for each new machine type using precision, recall, false-positive rate, precision-recall AUC, and expected maintenance cost.

Fourth, I would use time-aware evaluation and, where appropriate, hold out machines that were not included in training.

If the existing model performs adequately on the new machines, I would consider a controlled deployment. If it performs poorly, I would collect more representative examples and retrain or develop a machine-type-specific model.

Where failure examples are limited, the system should use conservative monitoring and expert inspection rather than assume that the model is reliable.

The company should document the validated machine types and avoid deploying the model beyond its demonstrated scope.

Q7. What fallback mechanism should be used if the prediction service becomes unavailable?

Answer

I would implement a safe fallback mechanism so that a temporary model-service outage does not leave the factory without a maintenance process.

The fallback strategy would include:

1. **Scheduled maintenance:** Continue the existing approved maintenance schedule until the predictive service becomes available again.
2. **Sensor-based alerts:** Retain independent alarms for critical sensor limits and known unsafe operating conditions.
3. **Local buffering:** Temporarily store sensor readings and timestamps so that data are not lost during the outage.
4. **Manual inspection:** Allow technicians to inspect critical machines according to established procedures.
5. **Service health monitoring:** Detect unavailable services, excessive response times, invalid predictions, and communication failures.
6. **Recovery and reconciliation:** Restore the service, validate incoming data, and reconcile buffered records without treating delayed predictions as if they were real-time warnings.
7. **Controlled recovery:** Verify the model service and its inputs before returning to normal predictive operation.

The fallback should not automatically shut down every machine solely because the prediction service is unavailable. Safety-critical shutdown decisions should follow approved industrial control procedures and machine-specific risk requirements.

This design ensures that the company continues to follow an established maintenance and safety process during service interruptions.

10. PROPOSED SYSTEM ARCHITECTURE

The proposed predictive maintenance system would contain the following stages.

Stage 1: Data collection

Sensors and machine controllers collect temperature, vibration, rotational speed, pressure, power consumption, and load measurements. Maintenance systems provide historical maintenance records and confirmed failure information.

Stage 2: Data preprocessing

The system checks data quality, handles missing values, identifies sensor errors, encodes machine types, and constructs time-based features.

Stage 3: Model training

Logistic Regression is trained as a baseline, and Gradient Boosting is evaluated as the advanced candidate. Training includes suitable class-imbalance handling and leakage-resistant validation.

Stage 4: Failure prediction

The trained model estimates the probability that a machine will fail within the next 24 hours.

Stage 5: Decision policy

The estimated probability is converted into an operational risk category. The system considers alert thresholds, machine criticality, intervention costs, and maintenance capacity.

Stage 6: Maintenance action

Technicians review the alert, inspect the machine, prepare spare parts, and carry out appropriate maintenance when justified.

Stage 7: Feedback and monitoring

Actual failures, inspection findings, and maintenance outcomes are recorded. These records are used to monitor performance and improve future model versions.

The model should be integrated with existing factory systems through a reliable service interface. Predictions should include timestamps, machine references, risk probabilities, and the model version used to generate them.

11. EXPECTED BENEFITS

The proposed predictive maintenance system is expected to provide the following benefits:

1. **Reduced unexpected failures:** Early warnings can help technicians intervene before a machine breaks down.
2. **Reduced downtime:** Better maintenance planning can prevent some production interruptions.
3. **Lower unnecessary maintenance costs:** Servicing can be prioritized according to machine condition and failure risk.
4. **Improved resource allocation:** Technicians and spare parts can be assigned to machines with greater urgency.
5. **Improved operational visibility:** Maintenance teams can monitor machine health and recurring failure patterns.
6. **Data-driven decisions:** Maintenance planning can be based on historical evidence, sensor measurements, and estimated risk.
7. **Continuous improvement:** Confirmed failure outcomes can be used to evaluate and update the model over time.

These benefits depend on the quality of the data, the model's performance, the effectiveness of maintenance interventions, and the company's ability to respond to alerts.

12. LIMITATIONS AND RISK MANAGEMENT

Although predictive maintenance offers significant potential, the proposed system has several limitations.

Data limitations: The dataset contains only 300 failure cycles out of 10,000 operating cycles. This may limit the model's ability to learn the full range of failure patterns.

Sensor reliability: Incorrect readings and calibration changes can produce misleading predictions.

Changing operating conditions: New machines, higher production loads, and new component defects may reduce model performance.

False positives: Excessive alerts may waste maintenance resources and reduce technician confidence in the system.

False negatives: Missed failures can still cause substantial downtime and financial losses.

Model interpretability: Complex models may require additional explanation tools to help technicians understand why an alert was generated.

Operational constraints: Maintenance teams may not have sufficient staff or spare parts to act on every alert immediately.

To manage these risks, the company should maintain reliable data-quality checks, use cost-sensitive evaluation, validate new machine types, monitor performance continuously, and retain an established fallback maintenance process.

13. CONCLUSION

This case study demonstrates how machine learning can improve maintenance planning in a smart manufacturing environment.

The company's existing fixed-schedule approach may result in unexpected failures and unnecessary maintenance. A predictive maintenance system can use machine operating data, sensor readings, and maintenance history to estimate the probability of failure within the next 24 hours.

The problem is best formulated as a supervised binary classification task. Reliable preprocessing, meaningful feature engineering, class-imbalance handling, and leakage-resistant validation are essential for building a dependable model.

Logistic Regression provides an interpretable baseline, while Gradient Boosting offers a more flexible approach for learning complex patterns in structured sensor data. Model evaluation must consider precision, recall, F1-score, false-positive rate, and expected operational cost rather than relying on accuracy alone.

In the provided evaluation, Model B detects more failures but produces more false-positive alerts. Its higher recall may make it preferable when preventing unexpected breakdowns is the primary objective, provided the additional maintenance burden is manageable.

The maintenance decision policy should balance failure risk, intervention costs, technician capacity, and spare-parts availability. After deployment, continuous monitoring, controlled retraining, validation on new machine types, and a safe fallback process are necessary to maintain reliability.

Overall, predictive maintenance can help the company reduce avoidable downtime, improve maintenance efficiency, and make better use of operational resources. Its success depends not only on predictive accuracy but also on reliable data, sound cost-based decisions, and effective integration with real manufacturing workflows.

Submitted by: Jui Tawde
Roll No.: 150096725149
Cohort: Sam Altman
Subject: Advanced Machine Learning Fundamentals
Date: 26 October 2026